INDIVIDUAL ASSIGNMENT 1

Simon Winther Albertsen

Department of Computer Science, University of Copenhagen

ABSTRACT

Recent image generation systems increasingly aim to combine text-to-image generation and instruction-based image editing within a single model. This report looks at FLUX.1 Kontext, a latent rectified-flow model that combines these tasks by conditioning on natural-language instructions and optional context images. The report describes its conditional formulation, latent representation, sequence-based image conditioning, and rectified flow-matching objective. The report also discusses InstructPix2Pix, a closely related earlier model for instruction-based image editing. Its synthetic data-generation pipeline based on GPT-3, Stable Diffusion, Prompt-to-Prompt, and CLIP is described and compared with FLUX.1 Kontext. The comparison shows a shift from specialized diffusion-based image editing toward transformer-based generative models that support both image generation and editing.

***Index Terms*—** Image generation, image editing, rectified flow matching, diffusion models, InstructPix2Pix

1. INTRODUCTION

Generative image models have developed from systems focused mainly on text-to-image generation toward models that can also change existing images using natural-language instructions. Image editing adds a challenge compared with unconstrained generation: the model must make the requested change while keeping relevant content from the input image. Earlier instruction-based approaches therefore often treated image editing as a separate task, using architectures or training methods designed for editing.

This report studies FLUX.1 Kontext, which instead treats image generation and image editing as two forms of the same conditional generation task. The model combines latent image representations, transformer-based processing, and rectified flow matching to support both text-to-image and image-to-image generation within a single architecture. To place this approach in context, the report also looks at InstructPix2Pix [1], an earlier instruction-based editing model. One of its main contributions is a method for creating a large synthetic editing dataset using pretrained language and image models. The two approaches are compared in terms of their training objectives, architectures, conditioning mechanisms, training data, and supported tasks.

2. FLUX.1 KONTEXT

FLUX.1 Kontext is a generative flow-matching model designed to combine image generation and image editing within a single architecture. Instead of relying on separate models or changes for specific tasks, such as adding LoRA weights, the model conditions generation jointly on an optional context image and a natural-language instruction. This allows a single model to support both image generation and many different image editing tasks.

2.1. Unified Image Generation and Editing

The central idea of FLUX.1 Kontext is to treat image generation and image editing as two forms of the same conditional generation task. Given a target image x, an optional context image y, and a natural-language instruction c, the model learns the conditional distribution [2]

$$p_\theta(x \mid y, c).$$

When no context image is given, $y = \emptyset$, the model performs standard text-to-image generation. When a context image is provided, the model instead performs image-to-image generation, covering both local editing and more generative in-context tasks.

This approach supports both small and large changes to an image. For local editing, the model can change a specific property or area while keeping the remaining content. For more generative in-context tasks, the context image instead provides a visual reference, such as a character, object, or style, that can be recreated in a very different scene. Thus, these tasks do not require separate architectures or changes to the model for each task, but are handled through different conditioning inputs to the same generative model.

2.2. Latent Representation and Conditioning

Rather than working directly in pixel space, FLUX.1 Kontext generates images in the latent space of an image autoencoder. High-resolution RGB images contain a large number of values, so processing them directly requires a lot of computation.

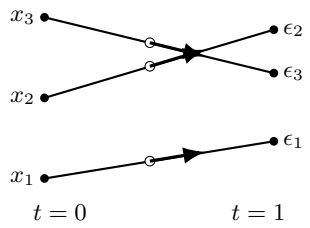


Fig. 1. Conditional training paths in rectified flow matching. Each sampled pair (x_i, ϵ_i) defines a straight interpolation path with constant conditional velocity $\epsilon_i - x_i$.

The autoencoder therefore compresses an image into a lower-dimensional latent representation, while a decoder converts the generated latent representation back into pixels to produce the target image.

FLUX.1 uses a convolutional autoencoder trained from scratch with 16 latent channels. Both the target image x and the context image y are encoded into sequences of latent image tokens. For conditioning, the context-image tokens are added to the end of the target-image token sequence, and the transformer processes all the tokens together. This simple sequence-concatenation strategy allows the model to use visual information from the reference image without requiring a separate image-conditioning architecture.

2.3. Rectified Flow Matching

This combined approach to generation and editing uses rectified flow. Given a clean target latent x and Gaussian noise $\epsilon \sim \mathcal{N}(0, I)$, FLUX.1 Kontext creates the linear interpolation

$$z_t = (1 - t)x + t\epsilon,$$

where $t \in [0, 1]$. At $t = 0$, the sample corresponds to the clean target latent $z_0 = (1 - 0)x + 0\epsilon = x$, while at $t = 1$ it corresponds to pure Gaussian noise $z_1 = (1 - 1)x + 1\epsilon = \epsilon$. Because the interpolation path is linear, the corresponding conditional target velocity is constant along the path and can be derived as

$$\frac{dz_t}{dt} = \frac{d}{dt}\big[(1 - t)x + t\epsilon\big] = \frac{d}{dt}\big[x - tx + t\epsilon\big] = \epsilon - x.$$

Figure 1 shows this construction for several sampled pairs (x_i, ϵ_i). Each pair defines a different straight interpolation path with the constant conditional velocity $\epsilon_i - x_i$.

The model is trained to predict this velocity from the noisy latent z_t by minimizing the squared L_2 distance between the predicted and target velocities, while conditioning on the context image y and the natural-language instruction c:

$$\mathcal{L}_\theta = \mathbb{E}\left[\|v_\theta(z_t, t, y, c) - (\epsilon - x)\|_2^2\right].$$

FLUX.1 Kontext samples the timestep t using a logit-normal shift schedule rather than uniformly over $[0, 1]$.

Although the conditional velocity $\epsilon - x$ is constant for a given sampled pair (x, ϵ), different pairs can produce similar values of z_t while having different target velocities. For example,

$$(x_1, \epsilon_1), \quad (x_2, \epsilon_2), \quad \dots$$

may produce the velocities

$$\epsilon_1 - x_1, \qquad \epsilon_2 - x_2, \qquad \dots$$

at similar points in latent space. The model observes z_t, t, y, and c, but not which exact pair (x, ϵ) produced the point. Under the squared-error objective, it therefore learns the expected velocity given the current point and conditioning,

$$v(z_t, t, y, c) = \mathbb{E}\left[\epsilon - x \mid z_t, t, y, c\right],$$

which corresponds to the conditional marginal velocity field.

At inference time, generation starts from Gaussian noise at $t = 1$. The learned velocity field defines the ordinary differential equation

$$\frac{dz_t}{dt} = v_\theta(z_t, t, y, c),$$

which is numerically integrated backwards from $t = 1$ toward $t = 0$. This gradually changes the noisy latent into the latent representation of a generated image while remaining conditioned on the given context image and natural-language instruction.

3. RELATED WORK: INSTRUCTPIX2PIX

InstructPix2Pix [1] is an early approach to instruction-based image editing. One of the paper's contributions is a large synthetic training dataset created using pretrained models. Rather than manually collecting hundreds of thousands of triplets made up of an input image, an editing instruction, and a corresponding edited image, Brooks et al. create these examples automatically using a combination of language and image generation models.

Manually producing such a dataset would be expensive, since each example would require describing an edit and generating a corresponding output image while keeping all image content unrelated to the requested change. InstructPix2Pix instead uses pretrained models to generate much of this supervision automatically.

3.1. Motivation and Relation to FLUX.1 Kontext

InstructPix2Pix showed that natural-language image editing could be learned from synthetically generated instruction-image pairs. FLUX.1 Kontext builds on the idea of editing images using instructions. It uses the same flow-matching model for both image generation and editing.

3.2. Method

The synthetic data-generation pipeline combines several pretrained models, each with a different role:

$$\underbrace{\text{GPT-3}}_{\text{generate edit}} \rightarrow \underbrace{\text{Stable Diffusion}}_{\text{render images}} \rightarrow \underbrace{\text{Prompt-to-Prompt}}_{\text{preserve structure}} \rightarrow \underbrace{\text{CLIP}}_{\text{filter pairs}}$$

3.2.1. GPT-3: Instruction Generation

The first stage of the pipeline creates text-based editing instructions. The goal is to teach a pretrained language model what counts as a reasonable image edit. Brooks et al. begin with captions sampled from LAION-Aesthetics and manually create 700 examples of the form

$$(c_{\text{before}},\ e,\ c_{\text{after}}),$$

where c_{before} is the original image caption, e is a natural-language editing instruction, and c_{after} describes what the image should look like after the edit. As an example for this report, the original caption "a photograph of me" could be paired with the instruction "make me stand beside LeBron James" and the edited caption "a photograph of me standing beside LeBron James."

At this stage, the examples contain no images; they describe the edits using only text. The 700 manually created examples are used to fine-tune GPT-3 Davinci for one epoch, so that the model learns the mapping

$$c_{\text{before}} \longmapsto (e, c_{\text{after}}).$$

This allows GPT-3 to generate new editing instructions and corresponding edited captions for previously unseen LAION captions. This small set of manually created examples lets GPT-3 generate hundreds of thousands of synthetic editing examples in text form. The edits include adding or removing objects and changing object properties, the setting, or the visual style.

This stage produces the text tuple $(c_{\text{before}}, e, c_{\text{after}})$. To train an image-editing model, InstructPix2Pix needs examples that include images, of the form

$$(x_{\text{before}},\ e,\ x_{\text{after}}).$$

Stable Diffusion and Prompt-to-Prompt generate these image pairs while keeping content unrelated to the requested edit.

3.2.2. Stable Diffusion and Prompt-to-Prompt

Given the text-based tuples produced by GPT-3, the next step is to create corresponding image pairs. A simple approach would be to generate x_{before} from the original caption and x_{after} from the edited caption using two separate Stable Diffusion generations. However, even small changes to a text prompt can lead to large changes in the generated image, such as a different person, pose, camera angle, background, or overall layout. This would give poor training examples, since the model should learn the requested edit rather than learn to replace the whole image with a different one.

To create image pairs that are more similar, Brooks et al. combine Stable Diffusion with Prompt-to-Prompt. The two images are generated using the same latent noise, and self-attention weights from the first generation are reused during an initial fraction p of the denoising process for the edited image. This helps the second image keep the same general structure and layout as the first image, while the edited caption controls what should change. The main idea is therefore to create image pairs that roughly follow

$$x_{\text{after}} \approx x_{\text{before}} + \text{requested edit},$$

instead of producing two mostly unrelated images.

The value of p controls how much of the original structure is kept. A larger p means that the edited image follows the structure of the original image for more of the denoising process, while a smaller p gives the model more freedom to make larger changes. Since different edits require different amounts of change, the authors generate several candidate image pairs using different values of p. These candidates are then filtered using CLIP to keep the image pairs that best match the requested edit.

3.2.3. CLIP Filtering

CLIP checks the quality of the generated image pairs and filters them. The authors mainly want to check three things. First, the before and after images should still be visually related. For example, adding LeBron James beside me should not also change the background. To measure this, they calculate the CLIP similarity between the two images and require

$$S_{\text{img-img}} > 0.75.$$

This helps ensure that most of the original image is kept.

Second, each generated image should match the caption used to generate it. Stable Diffusion may sometimes fail to follow the edited caption. For example, if the edited caption describes me standing beside LeBron James but the generated image still shows me alone, the pair is not useful for training. The authors therefore also require a minimum image–text CLIP similarity,

$$S_{\text{img-text}} > 0.2.$$

Finally, the authors use directional CLIP similarity to check whether the change between the two images matches the change described by the captions. They first calculate a text edit direction

$$\Delta_{\text{text}} = E_T(c_{\text{after}}) - E_T(c_{\text{before}}),$$

where E_T represents the CLIP text encoder. Similarly, they calculate the change between the image representations,

$$\Delta_{\text{image}} = E_I(x_{\text{after}}) - E_I(x_{\text{before}}),$$

where E_I is the CLIP image encoder. The similarity between these two changes is then measured using cosine similarity,

$$S_{\text{dir}} = \cos\left(\Delta_{\text{text}}, \Delta_{\text{image}}\right).$$

A high value means that the images changed in roughly the same semantic direction as the captions. For example, if the text changes from "a photograph of me" to "a photograph of me standing beside LeBron James", the image should add LeBron James beside me while keeping the rest of the scene similar. The authors require

$$S_{\text{dir}} > 0.2.$$

Together, these CLIP-based checks remove image pairs that are too different, do not match their captions, or do not correctly represent the requested edit.

3.3. Differences in Methods

Although InstructPix2Pix and FLUX.1 Kontext both perform instruction-based image editing, they use quite different methods.

First, InstructPix2Pix is based on Stable Diffusion and uses the standard diffusion objective. During training, noise is added to the latent representation of the target image, and the network is trained to predict this noise. FLUX.1 Kontext instead uses rectified flow matching, where the model is trained to predict the velocity

$$\epsilon - x$$

along the interpolation between the clean target latent and Gaussian noise.

The two models also use the input image differently. In InstructPix2Pix, the input image is encoded into latent space and concatenated with the noisy target latent along the channel dimension before being passed to the U-Net. In FLUX.1 Kontext, both the target and context images are represented as sequences of latent tokens. The context-image tokens are then added to the end of the target-image token sequence, and the transformer processes all the tokens together.

The models also differ in the tasks they can handle. InstructPix2Pix is designed specifically for image editing, where both an input image and an editing instruction are provided. FLUX.1 Kontext instead uses the same model for both image editing and image generation. When a context image is provided, it performs image-to-image generation, while removing the context image allows the same model to perform text-to-image generation.

Finally, the two papers differ in how they describe their training data. InstructPix2Pix creates its dataset synthetically using GPT-3, Stable Diffusion, Prompt-to-Prompt, and CLIP. FLUX.1 Kontext trains on millions of relational image pairs, but gives less detail about how these pairs were collected. It is therefore unclear whether its data avoids similar errors or biases.

4. DISCUSSION

InstructPix2Pix shows that instruction-based editing can be learned effectively using training examples generated by pretrained models, even when large amounts of paired editing data are not available. However, the training distribution depends on what the models used to create the dataset can produce. Errors or limited variety in the generated examples can carry over into the final editing model. InstructPix2Pix is designed specifically for image editing rather than general image generation. FLUX.1 Kontext uses a much more general conditional generative model trained on relational image data. It also replaces the diffusion U-Net with a transformer-based rectified-flow architecture. This broader approach still has limitations. FLUX.1 Kontext can still fail to follow instructions, and image quality can decrease when an image is edited repeatedly. The comparison therefore shows changes in both model design and the range of tasks the models support.

5. CONCLUSION

This report looked at FLUX.1 Kontext and compared it with InstructPix2Pix, an earlier model for instruction-based image editing. FLUX.1 Kontext combines image generation and editing in a single conditional model that uses rectified flow matching and a transformer working with latent representations. InstructPix2Pix focuses on image editing and uses a synthetic training dataset created with pretrained language and image models. The comparison shows a shift from systems designed specifically for instruction-based editing toward more general models that support both image generation and editing within a single architecture.

6. REFERENCES

[1] Tim Brooks, Aleksander Holynski, and Alexei A. Efros, "InstructPix2Pix: Learning to follow image editing instructions," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2023, pp. 18392–18402.

[2] Black Forest Labs, "Flux.1 kontext: Flow matching for in-context image generation and editing in latent space," *arXiv preprint arXiv:2506.15742*, 2025.